Assignment #2

1. Analyze and compare the performance of the Linear Search and Binary Search algorithms in the given searchSpace.txt for the words in wordSpace.txt

Date of Performance: 01/06/26

Date of Submission: 04/07/26

Student ID: 00724105101122

Name: Rohan Rahim

Group: C1

Objective:

The objective of this assignment is to analyze and compare the performance of Linear Search and Binary Search algorithms when searching for a set of target words within a dataset of 100,000 elements. Performance is evaluated based on execution time, energy consumption, memory usage, and estimated CO_2 emissions for Bangladesh.

Tasks:

1. Load a text array of size 100,000 (searchspace.txt) sorted in ascending alphabetical order.
2. Load a target array of size 100 (wordspace.txt) containing the words to search for.
3. Implement Linear Search
4. Implement Binary Search.
5. For each run, measure:
   - Execution time (s)
   - Estimated energy consumption (J)
     Energy (J) = CPU's approximate average power x Execution Time (s)
   - Peak memory usage (KB)
   - Estimated CO2 emissions (Bangladesh) Energy (kWh) = Energy (J) / 3,600,000
     CO2 (kg) = Energy (kWh) x 0.62

Experimental Results:

Algorithm	Data set	Total Words	Execution time(s)	Energy(J)	Peak memory (KB)	CO2 Emission(kg, Bd)
Linear Search	100,000	100	0.12987	1.2987	3680	0.00000022
BinarySearch	100,000	100	0.000209958	0.00209958	3664	0.00000000

Discussion:

● **Linear Search :** Linear Search examines each element of the array sequentially regardless it's sorted or not from the beginning until it encounters the target word or hits the end of the data space. When looking up a batch of 100 words across a substantial dataset of 100,000 strings, it frequently iterates deep into the array. This repetitive, element-by-element scanning forces it to perform millions of string comparisons, resulting in a noticeably high execution time of **0.12987 seconds** and a correspondingly higher energy footprint of **1.2987 Joules**.

● **Binary Search:** Binary Search relies heavily on the fact that the search space is sorted in ascending alphabetical order. Instead of checking every item, it repeatedly divides the active search range in half. For a 100,000-word dataset, narrowing down any single target word takes at most approximately 17 split comparisons ($\log_2$ 100,000 which is approximately 16.61). Because it eliminates massive blocks of data at every single step, it successfully looks up the entire target batch in a mere **0.000209958 seconds** consuming negligible energy (**0.00209958 Joules**) and registering virtually zero measurable CO_2 emissions.

Overall Summary:

The results clearly show that Binary Search is much faster and more efficient than Linear Search. Because the array was already sorted, Binary Search was able to cut the search space in half with every single step, drastically reducing the time needed to find the words. Linear Search, on the other hand, had to check the words one by one, which took a lot more time and used more energy. While both algorithms used almost the same amount of memory to hold the words, Binary Search is the far better choice for large, sorted datasets.

Code:

```
#include <iostream>
#include <chrono>
#include <vector>
#include <string>
#include <fstream>

using namespace std;

// OS-Specific Headers for Memory Tracking
#if defined(_WIN32)
  #include <windows.h>
  #include <psapi.h>
#elif defined(__APPLE__)
  #include <mach/mach.h>
#elif defined(__linux__)
  #include <sys/resource.h>
  #include <fstream>
  #include <unistd.h>
#endif

// Constants
constexpr double CPU_POWER_WATTS = 10.0;
constexpr double BD_EMISSION_FACTOR = 0.62; // kg CO₂ / kWh

// Cross-platform function to get Peak RSS (Resident Set Size) in KB
size_t getPeakMemoryKB() {
#if defined(_WIN32)
  PROCESS_MEMORY_COUNTERS pmc;
  if (GetProcessMemoryInfo(GetCurrentProcess(), &pmc, sizeof(pmc))) {
    return pmc.PeakWorkingSetSize / 1024;
  }
#elif defined(__APPLE__)
  struct mach_task_basic_info info;
  mach_msg_type_number_t count = MACH_TASK_BASIC_INFO_COUNT;
  if (task_info(mach_task_self(), MACH_TASK_BASIC_INFO, (task_info_t)&info, &count) == KERN_SUCCESS) {
    return info.resident_size_max / 1024; // macOS returns max bytes
  }
#elif defined(__linux__)
  struct rusage usage;
  if (getrusage(RUSAGE_SELF, &usage) == 0) {
    return usage.ru_maxrss; // Linux returns maxrss in KB directly
  }
#endif
  return 0;
}


vector<string> loadFile(const string& filename) {
  ifstream file(filename);
  vector<string> words;

  if (!file.is_open()) {
    cerr << "Error: Could not open file " << filename << endl;
    return words;
  }

  int count = 0;
  if (file >> count) {
    // Clear the leftover newline character right after the number 100
    string dummy;
    getline(file, dummy);

    // Reserve memory space for efficiency
    words.reserve(count);

    string word;
    while (getline(file, word)) {
      // Only add lines that actually contain characters
```

```
        if (!word.empty()) {
            words.push_back(word);
        }
    }
  }
  return words;
}

// ----- Algorithms --------- //
int linearSearch(vector<string>& searchSpace, int n, string word) {
  for(int i = 0; i < n; i++) {
    if(searchSpace[i] == word) {
      return 1;
    }
  }
  return -1;
}

int binarySearch(vector<string>& searchSpace, int n, string word)
{
  int low = 0, high = n-1;

  while(low <= high) {
    int mid = (low+high)/2;
    if (searchSpace[mid] == word) {
      return 1;
    } else if(searchSpace[mid] > word) {
      high = mid-1;
    } else {
      low = mid + 1;
    }
  }

  return -1;
}


int main() {
  // Read from both files
  vector<string> searchSpace = loadFile("searchspace.txt");
  vector<string> wordSpace = loadFile("wordspace.txt");

  int n = searchSpace.size();

  // Verify the files loaded successfully
  if (searchSpace.empty() || wordSpace.empty()) {
    cerr << "Error loading files. Check if they exist in the correct
directory.\n";
    return 1;
  }

  cout << "Successfully loaded " << searchSpace.size() << " words
into searchSpace.\n";
  cout << "Successfully loaded " << wordSpace.size() << " words
into wordSpace.\n";


  // High-resolution cross-platform timer
  auto start = chrono::high_resolution_clock::now();

  // -------- Your algorithm here --------
  for (string word: wordSpace) {
    // linearSearch(searchSpace, n, word);
    binarySearch(searchSpace, n, word);
  }
  // ---------------------------------------

  auto end = chrono::high_resolution_clock::now();
  chrono::duration<double> elapsed = end - start;

  // Calculations
  double elapsed_seconds = elapsed.count();

  double energy = CPU_POWER_WATTS * elapsed_seconds; //
Joules
  double energy_kWh = energy / 3.6e6;              // kWh
  double co2 = energy_kWh * BD_EMISSION_FACTOR;    // kg
CO₂

  // Print Results
  cout << "Execution time: " << elapsed_seconds << " seconds\n";
  cout << "Estimated energy consumption: " << energy << "
Joules\n";

  size_t peakMem = getPeakMemoryKB();
  if (peakMem > 0) {
    cout << "Peak Memory Usage: " << peakMem << " KB\n";
  } else {
    cout << "Failed to get memory info.\n";
  }

  // Using printf-style formatting for scientific precision of CO2
  printf("Estimated CO2 emissions (Bangladesh): %.8f kg\n", co2);

  return 0;
}
```